

Lecture 7: X-Hour Text Classification Workshop

Week 2: Hands-On Machine Learning for NLP

PSYC 51.07: Models of Language and Communication

Learning Objectives

By the end of this session, you will:

1. Build text classifiers from scratch using scikit-learn
2. Understand different text representation methods (BoW, TF-IDF, embeddings)
3. Compare Naive Bayes, Logistic Regression, and Neural approaches
4. Evaluate classifier performance using appropriate metrics
5. Debug common issues in text classification pipelines

Workshop format: Hands-on coding with the 20 Newsgroups dataset

Workshop Overview

Today's Agenda:

1. **Part 1:** Loading and exploring real data
2. **Part 2:** Feature engineering for text (BoW, TF-IDF)
3. **Part 3:** Building classifiers (Naive Bayes, Logistic Regression, Neural Networks)
4. **Part 4:** Model comparison and analysis
5. **Part 5:** Error analysis and improvements
6. **Part 6:** Real-world considerations (class imbalance)

Companion notebook: `xhour_classification_demo.ipynb`

Part 1: The 20 Newsgroups Dataset

A classic text classification benchmark:

- Posts from 20 different newsgroups
- ~20,000 documents total
- Good for learning classification fundamentals

Today's subset (4 categories):

- `sci.space` - Science discussions about space
- `rec.sport.hockey` - Sports discussions about hockey
- `talk.politics.misc` - Political discussions
- `comp.graphics` - Computer graphics

Why these? Relatively distinct topics for easier learning.

Loading the Data

```
1 from sklearn.datasets import fetch_20newsgroups
2
3 categories = [
4     'sci.space',
5     'rec.sport.hockey',
6     'talk.politics.misc',
7     'comp.graphics'
8 ]
9
10 train_data = fetch_20newsgroups(
11     subset='train',
12     categories=categories,
13     shuffle=True,
14     random_state=42,
15     remove=('headers', 'footers', 'quotes') # Remove metadata
16 )
17
18 print(f"Loaded {len(train_data.data)} training documents")
```

Exploring the Data

Key questions to ask:

1. How many documents per category?
2. What do the documents look like?
3. What words/phrases might be good indicators?
4. Are there any categories that might be hard to distinguish?

Data exploration is critical before modeling!

Always look at your data before building models.

Exploring the Data: Concrete Example

```
1import pandas as pd
2from collections import Counter
3
4# Check class distribution
5print("Documents per category:")
6for i, name in enumerate(train_data.target_names):
7    count = (train_data.target == i).sum()
8    print(f" {name}: {count}")
9
10# Output:
11#   sci.space: 593
12#   rec.sport.hockey: 600
13#   talk.politics.misc: 465
14#   comp.graphics: 584
15
16# Look at a sample document
17print("\n--- Sample document (sci.space) ---")
18idx = [i for i, t in enumerate(train_data.target) if t == 0][0]
```

Exploring the Data: Concrete Example

```
21# Output might show:  
22# "NASA announced today that the Mars rover has discovered  
23#  evidence of water ice beneath the surface..."
```

...continued

Notice: Classes are roughly balanced (good!), but `talk.politics.misc` has fewer examples.

Part 2: Feature Engineering

The fundamental question:

How do we convert text to numbers for machine learning?

Three approaches today:

1. **Bag of Words (BoW):** Count word frequencies
2. **TF-IDF:** Weight by document frequency
3. **Dense embeddings:** (Preview for future lectures)

Method 1: Bag of Words (BoW)

The simplest approach: Count how many times each word appears.

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 bow_vectorizer = CountVectorizer(
4     max_features=5000,      # Keep only top 5000 words
5     min_df=2,              # Word must appear in at least 2 docs
6     max_df=0.8,            # Word must appear in <80% of docs
7     stop_words='english'   # Remove common words
8 )
9
10 X_train_bow = bow_vectorizer.fit_transform(train_data.data)
```

Result: Sparse matrix of word counts

Bag of Words: Limitations

What BoW captures:

- Word presence/frequency
- Vocabulary overlap between documents

What BoW ignores:

- Word order ("not good" vs "good not")
- Semantics ("great" vs "excellent")
- Context

Key insight: Common words dominate but are often uninformative!

BoW: Concrete Vector Example

What does a BoW vector actually look like?

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 docs = [
4     "NASA launches rocket to Mars",
5     "Hockey game ends in overtime",
6     "NASA discovers water on Mars"
7 ]
8
9 vectorizer = CountVectorizer()
10 X = vectorizer.fit_transform(docs)
11
12 # Vocabulary mapping
13 print("Vocabulary:", vectorizer.vocabulary_)
14 # {'nasa': 5, 'launches': 4, 'rocket': 7, 'to': 8, 'mars': 6,
15 #   'hockey': 2, 'game': 1, 'ends': 0, 'in': 3, 'overtime': 9,
16 #   'discovers': 10, 'water': 11, 'on': 12}
```

BoW: Concrete Vector Example

21# └ "launches"=1, "nasa"=1, "mars"=1, "rocket"=1, "to"=1

...continued

Observation: Most entries are 0 (sparse!). Documents share "mars" and "nasa".

Method 2: TF-IDF

Term Frequency-Inverse Document Frequency

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

where:

- $\text{TF}(t, d)$ = frequency of term t in document d
- $\text{IDF}(t) = \log \frac{N}{\text{df}(t)}$ = inverse document frequency

Intuition: Downweight common words, upweight rare informative words!

TF-IDF in Practice

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 tfidf_vectorizer = TfidfVectorizer(
4     max_features=5000,
5     min_df=2,
6     max_df=0.8,
7     stop_words='english',
8     use_idf=True,
9     sublinear_tf=True # Use log scaling for term frequency
10 )
11
12 X_train_tfidf = tfidf_vectorizer.fit_transform(train_data.data)
```

Result: Sparse matrix of TF-IDF scores

BoW vs TF-IDF Comparison

Same document, different representations:

Word	BoW Count	TF-IDF Score
"the"	15	0.02 (low - common everywhere)
"nasa"	3	0.45 (high - rare, informative)
"space"	5	0.38 (moderate - distinctive)

TF-IDF identifies the truly distinctive terms!

Part 3: Building Classifiers

Three approaches:

1. **Naive Bayes:** Fast, probabilistic, good baseline
2. **Logistic Regression:** Linear, interpretable, often best
3. **Neural Network:** Flexible, can learn complex patterns

Which will win? Let's find out!

Classifier 1: Naive Bayes

Bayes' theorem with independence assumption:

$$P(y|x) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Why "naive"? Assumes features are independent (they're not!)

Why does it work? Despite the wrong assumption, it often performs well for text.

```
1 from sklearn.naive_bayes import MultinomialNB
2
3 nb = MultinomialNB()
4 nb.fit(X_train_tfidf, train_data.target)
```

Classifier 2: Logistic Regression

Learns weights for each feature:

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

Advantages:

- Interpretable weights (which words matter?)
- Often outperforms Naive Bayes
- Fast training and prediction

```
1 from sklearn.linear_model import LogisticRegression
2
3 lr = LogisticRegression(max_iter=1000, C=1.0)
4 lr.fit(X_train_tfidf, train_data.target)
```

Analyzing Feature Weights

Logistic Regression gives us interpretable weights!

Category	Top Positive Features
sci.space	nasa, orbit, shuttle, moon, launch
rec.sport.hockey	hockey, nhl, team, game, play
talk.politics.misc	government, president, tax, policy
comp.graphics	image, graphics, 3d, rendering

The model learns what we'd expect!

Classifier 3: Simple Neural Network

Feedforward architecture:

1Input (TF-IDF) -> Hidden Layer 1 (256) -> Hidden Layer 2 (128) -> Output (4 cla

```
1import torch.nn as nn
2
3class TextClassifier(nn.Module):
4    def __init__(self, input_dim, hidden_dim, output_dim):
5        super().__init__()
6        self.fc1 = nn.Linear(input_dim, hidden_dim)
7        self.fc2 = nn.Linear(hidden_dim, hidden_dim // 2)
8        self.fc3 = nn.Linear(hidden_dim // 2, output_dim)
9        self.dropout = nn.Dropout(0.3)
10        self.relu = nn.ReLU()
```

Part 4: Model Comparison

Model	Accuracy
Naive Bayes (BoW)	~85%
Naive Bayes (TF-IDF)	~87%
Logistic Regression	~90%
Neural Network	~89%

Key insight: For bag-of-words features, linear models are competitive!

Neural networks shine with richer representations (embeddings).

Evaluation Metrics

Beyond accuracy:

- **Precision:** Of predicted positives, how many are truly positive?
- **Recall:** Of actual positives, how many did we catch?
- **F1-Score:** Harmonic mean of precision and recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why not just accuracy?

- Imbalanced datasets
- Different costs for different errors

Confusion Matrix

Visual representation of classifier errors:

	Predicted A	Predicted B	Predicted C	Predicted D
Actual A	85	2	3	0
Actual B	1	92	2	5
Actual C	4	3	88	5
Actual D	0	8	2	90

Diagonal = correct predictions. Off-diagonal = errors.

Part 5: Error Analysis

Critical step often skipped!

1. Find misclassified examples
2. Look for patterns
3. Understand why the model failed
4. Use insights to improve

Example errors to examine:

- Documents with mixed topics
- Short documents with little signal
- Unusual vocabulary

Error Analysis: Concrete Example

```
1# Find misclassified examples
2y_pred = lr.predict(X_test_tfidf)
3errors = np.where(y_pred != test_data.target)[0]
4
5print(f"Found {len(errors)} misclassifications out of {len(y_pred)}")
6
7# Examine a specific error
8idx = errors[0]
9print(f"\nMisclassified document:")
10print(f"  True: {test_data.target_names[test_data.target[idx]]}")
11print(f"  Predicted: {test_data.target_names[y_pred[idx]]}")
12print(f"\nText preview:")
13print(test_data.data[idx][:300])
```

Example output:

Common Error Patterns

Why do classifiers fail?

1. **Ambiguous content:** Document mentions multiple topics
2. **Limited context:** Very short documents
3. **Domain shift:** Test data differs from training
4. **Rare vocabulary:** Important words not in training

Solution ideas:

- Better preprocessing
- More features (bigrams, trigrams)
- Domain-specific fine-tuning

Part 6: Class Imbalance

What if some classes have many more examples?

Problem: Model learns to predict majority class

Solutions:

1. **Class weights:** Penalize errors on minority classes more
2. **Oversampling:** Duplicate minority class examples
3. **Undersampling:** Remove majority class examples
4. **SMOTE:** Generate synthetic minority examples

```
1lr_balanced = LogisticRegression(  
2    class_weight='balanced'  # Automatically adjust weights  
3)
```

Discussion Questions

1. **BoW vs TF-IDF:** When would you prefer one over the other?
2. **Linear vs Neural:** Why didn't the neural network significantly outperform logistic regression?
3. **Feature Engineering:** How important was feature engineering compared to model choice?
4. **Scalability:** Which approach scales best to millions of documents?
5. **Interpretability:** Which models are most interpretable? Why does it matter?

Key Takeaways

1. **Good features matter more than complex models** (for many tasks)
2. **TF-IDF usually beats raw BoW** for text classification
3. **Linear models are competitive** with neural networks on bag-of-words
4. **Always examine errors** to understand model behavior
5. **Consider class imbalance** and adjust accordingly

Connection to Course Themes

This week's pipeline:

1Data Cleaning -> Tokenization -> Feature Extraction -> Classification
2(Lecture 5) (Lecture 6) (Today) (Today)

Next lecture: POS Tagging & Sentiment Analysis

How do these building blocks combine for real NLP applications?

Hands-On Exercise

Open the companion notebook: `xhour_classification_demo.ipynb`

1. Load the 20 Newsgroups dataset
2. Experiment with different vectorizers
3. Train multiple classifiers
4. Analyze errors and improve
5. Try your own text examples!

Goal: Build intuition for text classification

Additional Resources

Libraries:

- scikit-learn: <https://scikit-learn.org/>
- PyTorch: <https://pytorch.org/>

Datasets:

- 20 Newsgroups: Classic benchmark
- IMDb Reviews: Sentiment classification
- AG News: News categorization

HuggingFace:

- [Chapter 1: Transformer Models](#)

Questions?

Next: Lecture 8 - POS Tagging & Sentiment Analysis